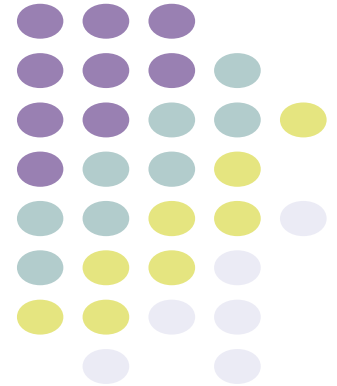
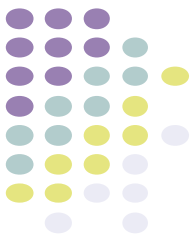


Redes de Computadores

Prof. Flavio Barbieri Gonzaga

flavio.gonzaga@unifal-mg.edu.br

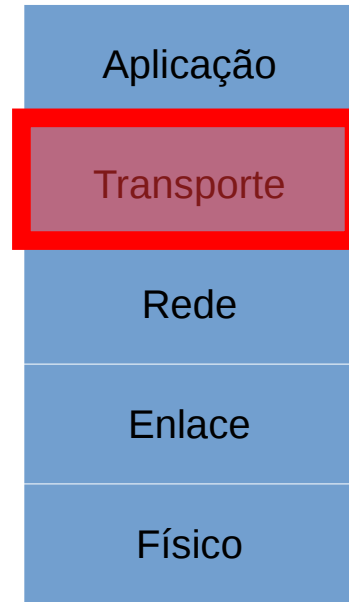
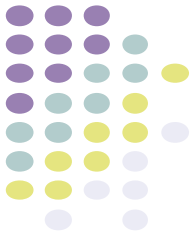




Sumário

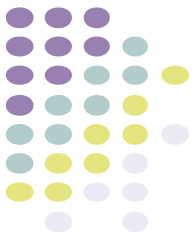
- Camada de Transporte
 - Transferência confiável de dados
 - Go-Back-N
 - Selective Repeat

Camada de Transporte

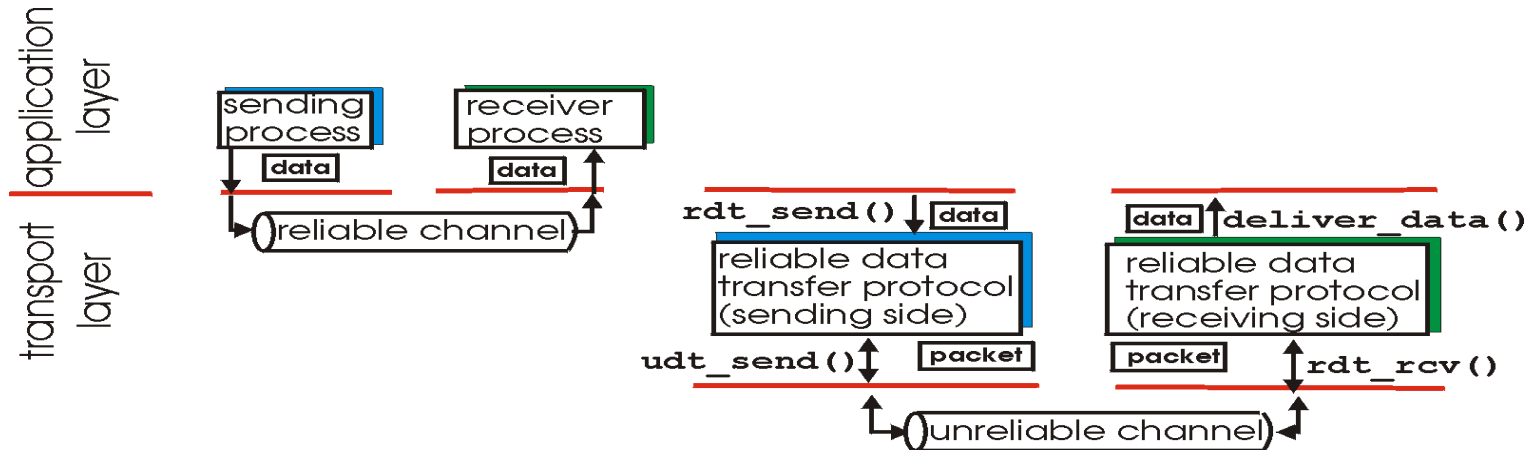


Pilha de protocolos da Internet
(cinco camadas)

Transferência Confiável de Dados



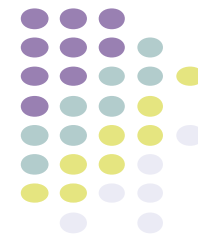
- Camada que implementa a transferência confiável dos dados
- Característica não-confiável do canal influenciará na complexidade do protocolo de transferência confiável dos dados



(a) provided service

(b) service implementation

Transferência Confiável de Dados

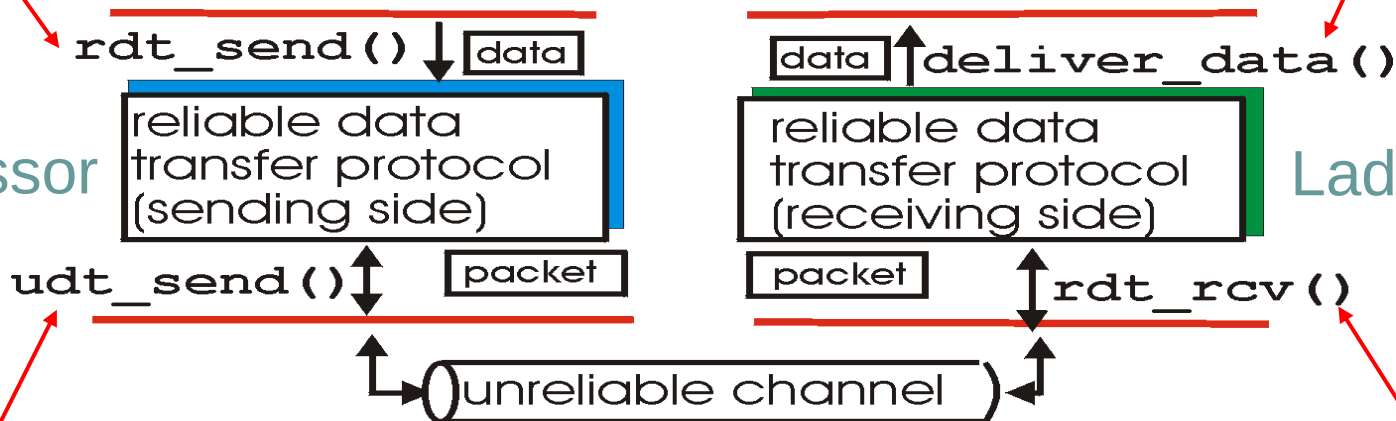


`rdt_send()`: chamado de cima, (ex. Aplicação).
Envia dados a serem passados à camada seguinte.

`deliver_data()`: chamado pelo rdt para entregar dados.

Lado emissor

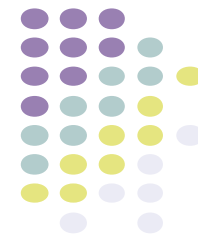
Lado receptor



`udt_send()`: chamado pelo rdt, para enviar pacotes pelo canal não confiável ao receptor.

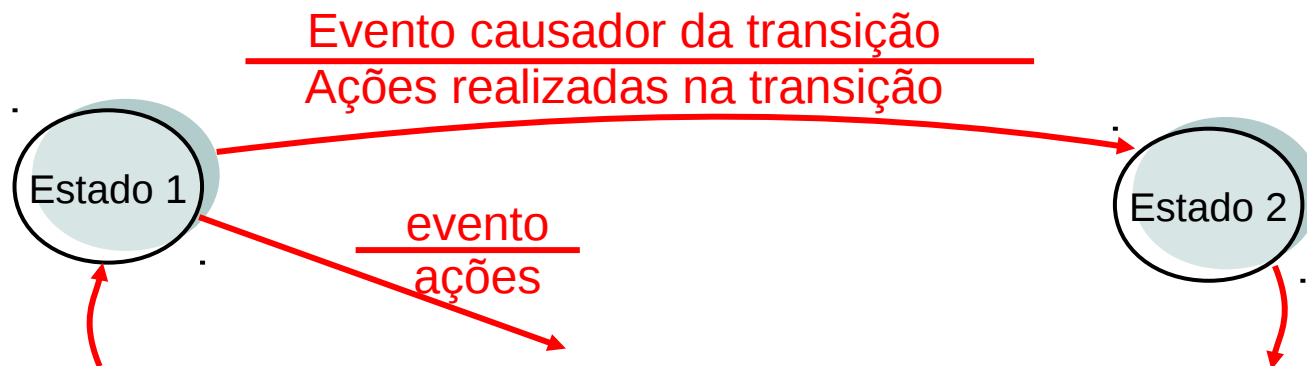
`rdt_rcv()`: chamado quando o pacote chega ao rdt do destinatário.

Transferência Confiável de Dados

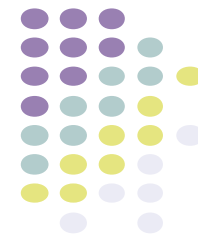


Desenvolvimento do rdt:

- Será feito em partes, até que tenhamos um protocolo de transferência confiável
- Dados fluem em um único sentido
 - Informações de controle fluem em ambos os sentidos
- Utilização de máquinas de estado finito para definir emissor e destinatário

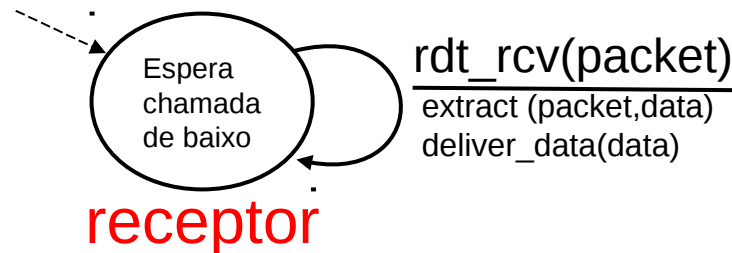
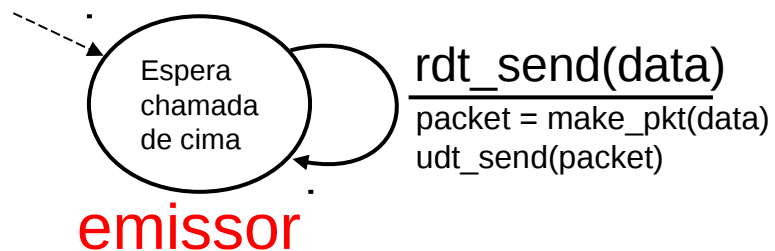


Transferência Confiável de Dados

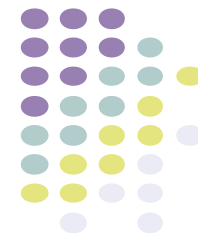


rdt 1.0

- Canal totalmente confiável:
 - Sem perda de pacotes
 - Sem erro de bits
- Emissor envia dados pelo canal
- Receptor recebe dados pelo canal



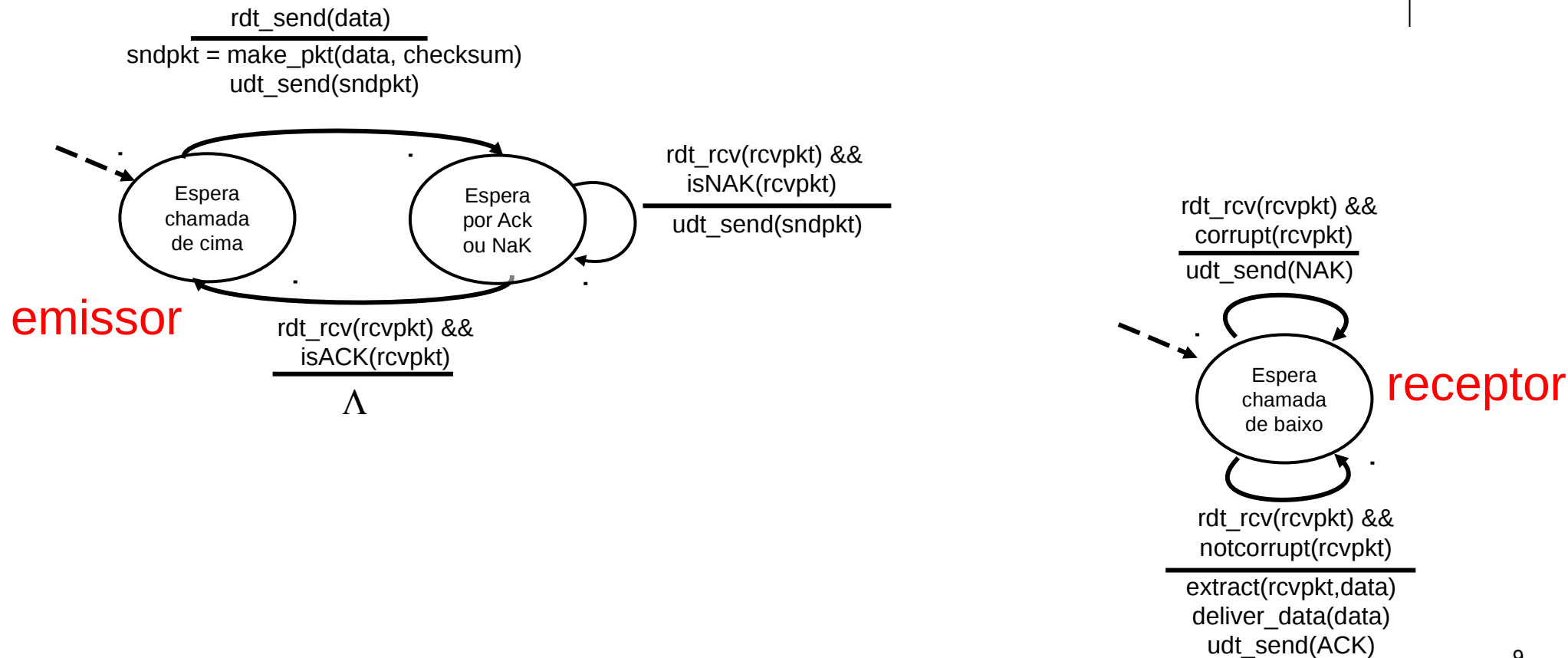
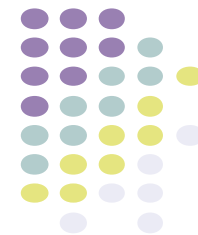
Transferência Confiável de Dados



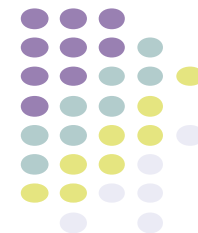
rdt 2.0

- Canal pode inverter bits
 - Cheksum para verificar erros de bits
- Como recuperar dos erros?
 - Acknowledgements (ACKs): receptor informa que o pacote foi corretamente recebido (OK)
 - Negative acknowledgements (NAKs): receptor informa que o pacote foi recebido com erro
 - Emissor retransmite pacote ao receber NAK
- Novidades do rdt 2.0:
 - Detecção de erros
 - Feedback do receptor: mensagens de controle rcvr → sender

Transferência Confiável de Dados



Transferência Confiável de Dados

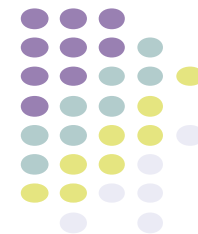


rdt 2.0

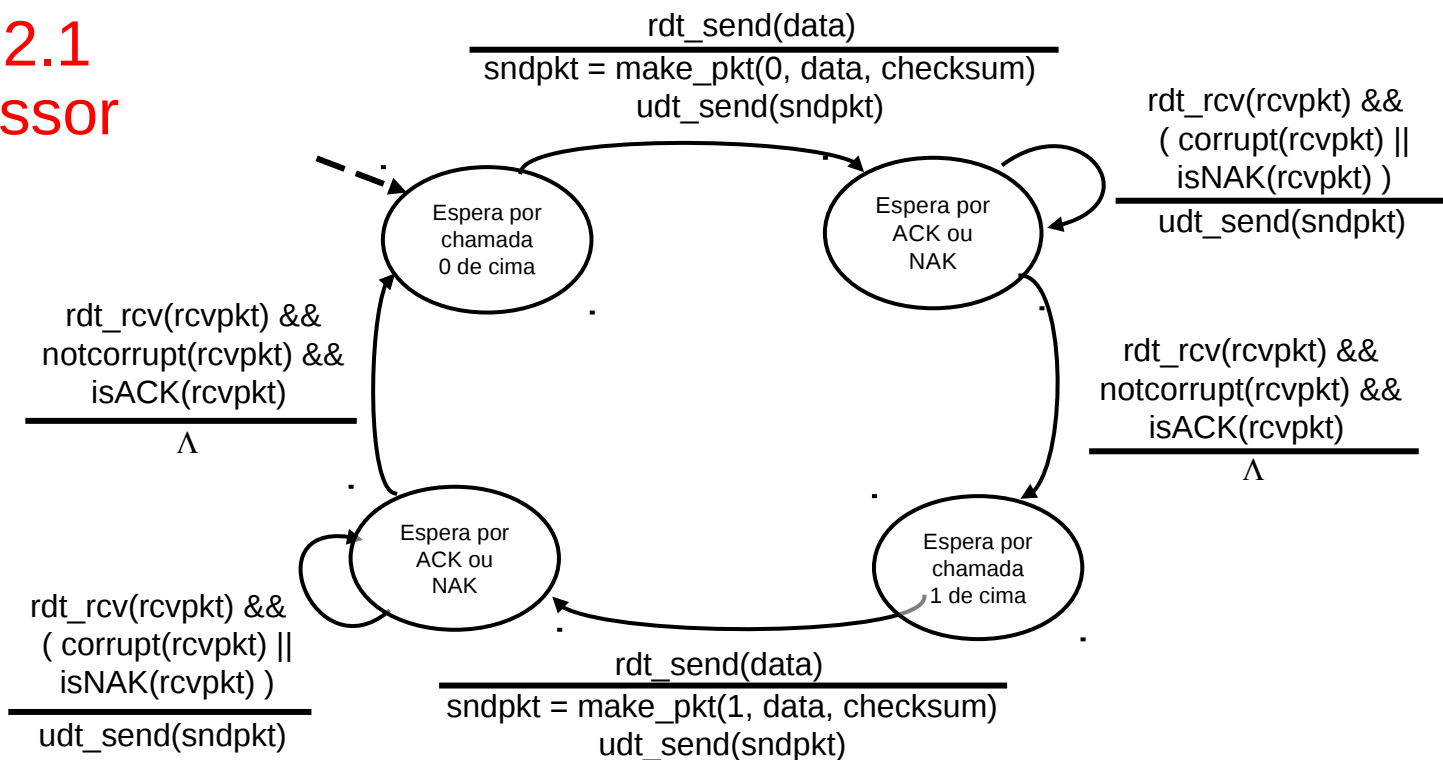
- Problema!
 - O que acontece caso o pacote ACK ou NAK seja corrompido?
 - Emissor não saberá o que aconteceu no receptor
 - Não pode apenas retransmitir, uma vez que poderá enviar pacotes duplicados!
- Tratando pacotes duplicados: rdt 2.1
 - Emissor acrescenta números de sequência à cada pacote
 - Emissor retransmite pacote atual, caso ACK ou NAK chegue corrompido
 - Receptor descarta pacotes duplicados

stop and wait
Emissor envia um pacote,
então espera pela resposta
do receptor.

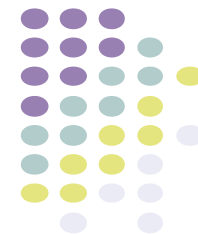
Transferência Confiável de Dados



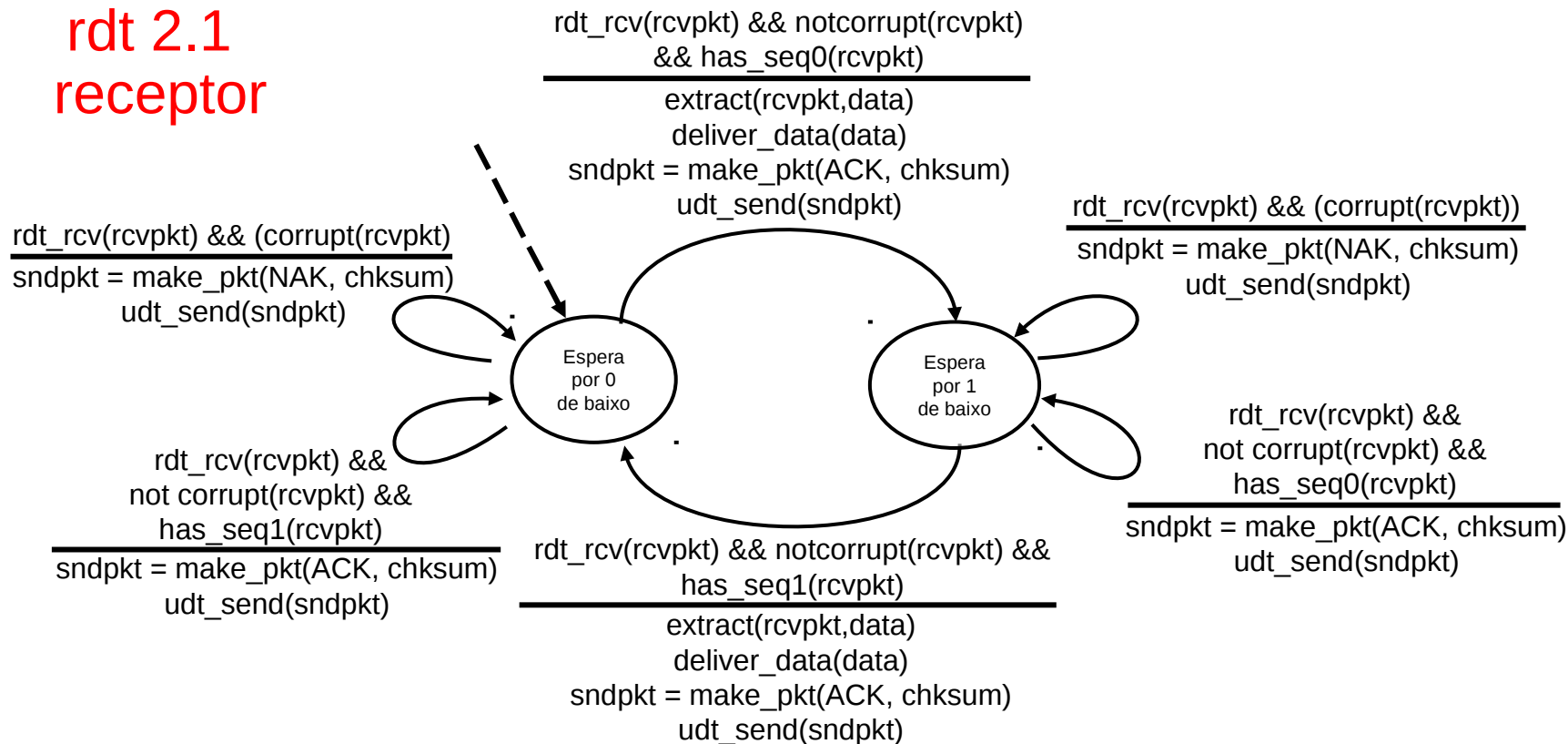
rdt 2.1 emissor



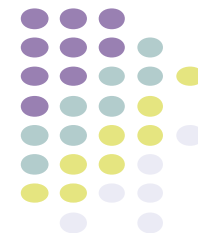
Transferência Confiável de Dados



rdt 2.1 receptor



Transferência Confiável de Dados

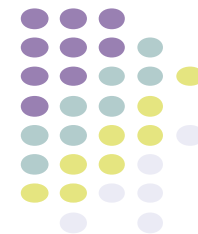


rdt 2.1

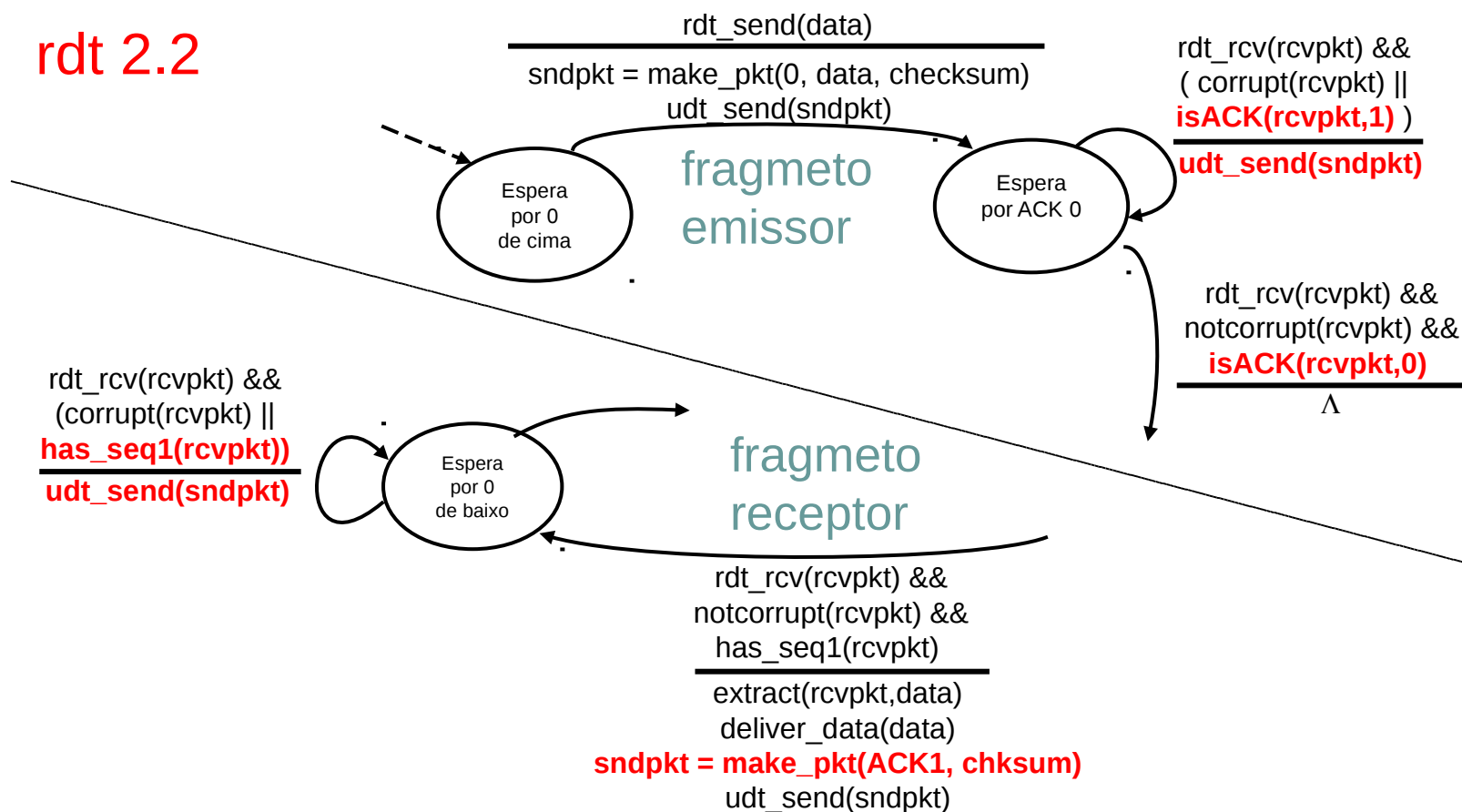
- Utiliza de NAKs e ACKs
- NAK pode ser substituído por ACK para o último pacote corretamente recebido
- ACKs duplicados significam pacotes perdidos (destinatário não recebeu o pacote seguinte àquele cujo ACK foi duplicado)!

rdt 2.2

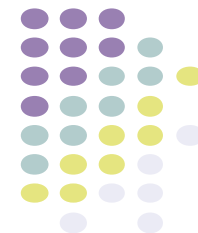
Transferência Confiável de Dados



rdt 2.2



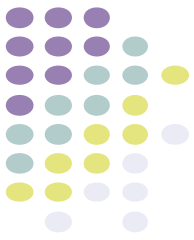
Transferência Confiável de Dados



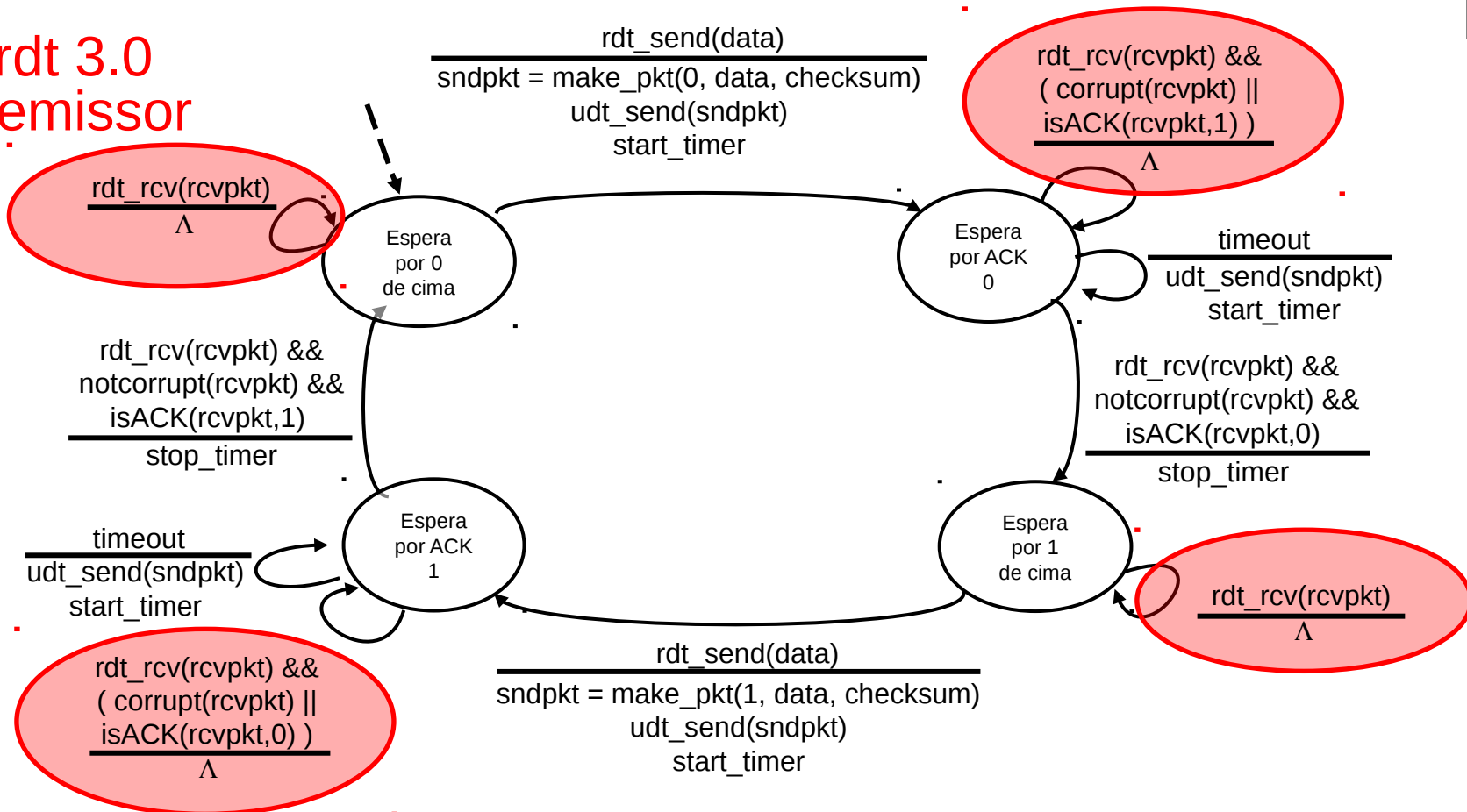
rdt 3.0

- Canal com erro e perda de pacotes:
 - Emissor espera por um tempo pelo ACK
 - Caso não receba, retransmite pacote
 - Se o pacote ou ACK estiver apenas atrasado, mas não perdido?
 - Retransmissão será duplicada, mas o número de sequência já trata esse problema!
 - Receptor deve especificar o número de sequência do pacote que está sendo confirmado (ACK)

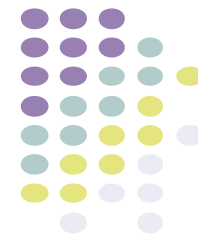
Transferência Confiável de Dados



rdt 3.0
emissor



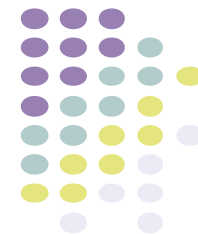
Transferência Confiável de Dados



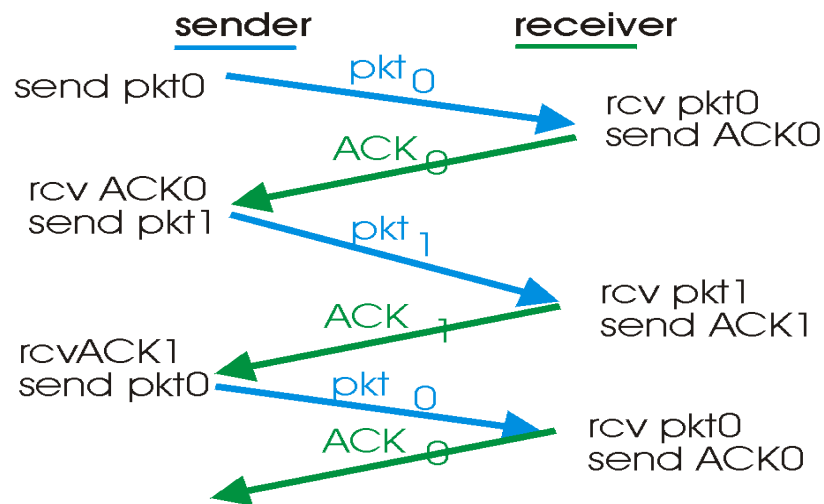
rdt 3.0
receptor?

Exercício!

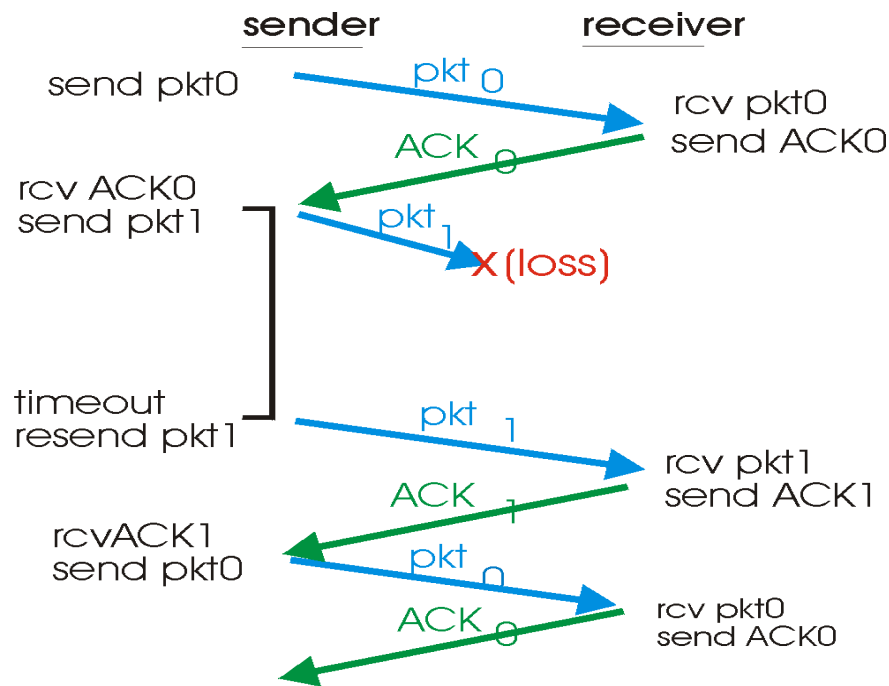
Transferência Confiável de Dados



rdt 3.0

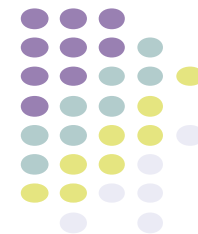


(a) operation with no loss

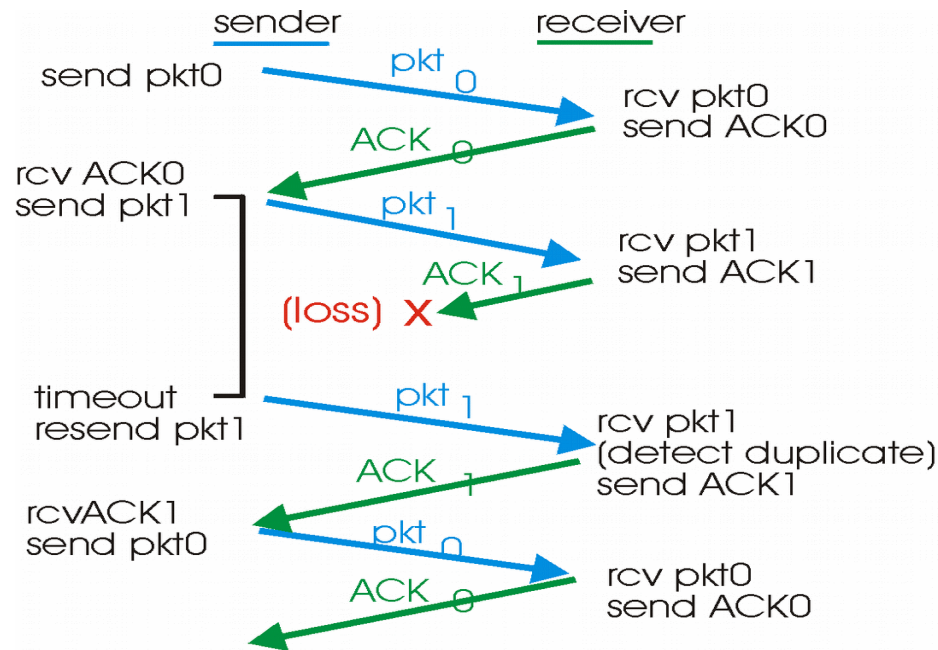


(b) lost packet

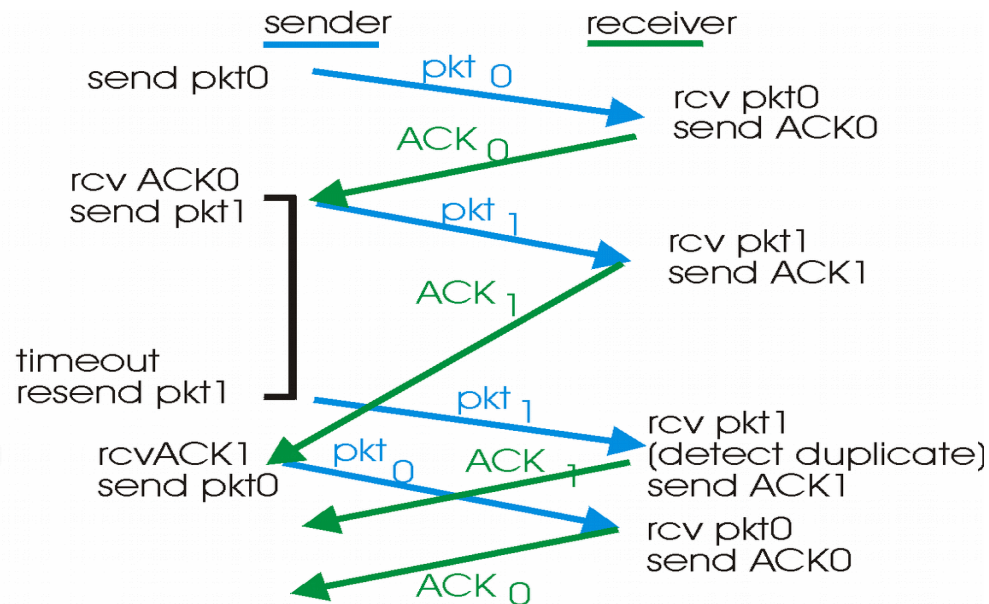
Transferência Confiável de Dados



rdt 3.0

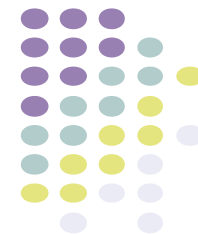


(c) lost ACK



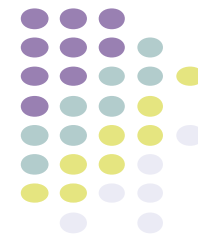
(d) premature timeout

Transferência Confiável de Dados



- rdt 3.0 funciona, mas sua performance é muito ruim;
- Exemplo:
 - Link = 1Gbps
 - Pacote 1KB
 - Atraso de propagação fim a fim de 15 ms

Transferência Confiável de Dados



- rdt 3.0 funciona, mas sua performance é muito ruim;

$$T_{\text{transm}} = \frac{L \text{ (tamanho pacote em bits)}}{R \text{ (taxa transm, bps)}} = \frac{8\text{kb/pkt}}{10^{**9} \text{ b/seg}} = 8 \text{ microssegundos}$$

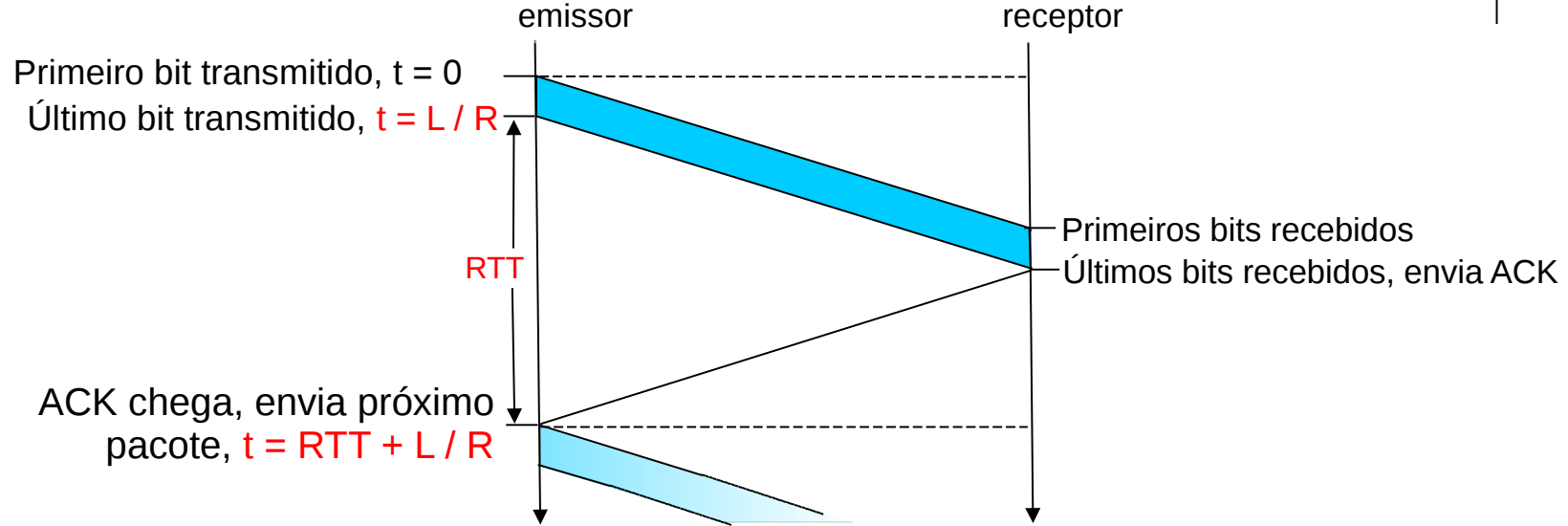
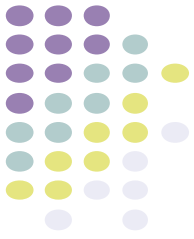
$$U_{\text{emissor}} = \frac{L / R}{RTT + L / R} = \frac{.008}{30.008} = 0.00027$$

U_{emissor} Fração de tempo que o emissor permanece ativo enviando dados

Múltiplo	Equivalente decimal	Nome	Símbolo
10 ⁻³	0,001	milissegundo	ms
10 ⁻⁶	0,000001	microssegundo	µs
10 ⁻⁹	0,000000001	nanossegundo	ns
10 ⁻¹²	0,000000000001	picossegundo	ps
10 ⁻¹⁵	0,000000000000001	fentossegundo	fs
10 ⁻¹⁸	0,000000000000000001	atossegundo	as
10 ⁻²¹	0,000000000000000000001	zeptossegundo	zs
10 ⁻²⁴	0,00000000000000000000001	yoctossegundo	ys

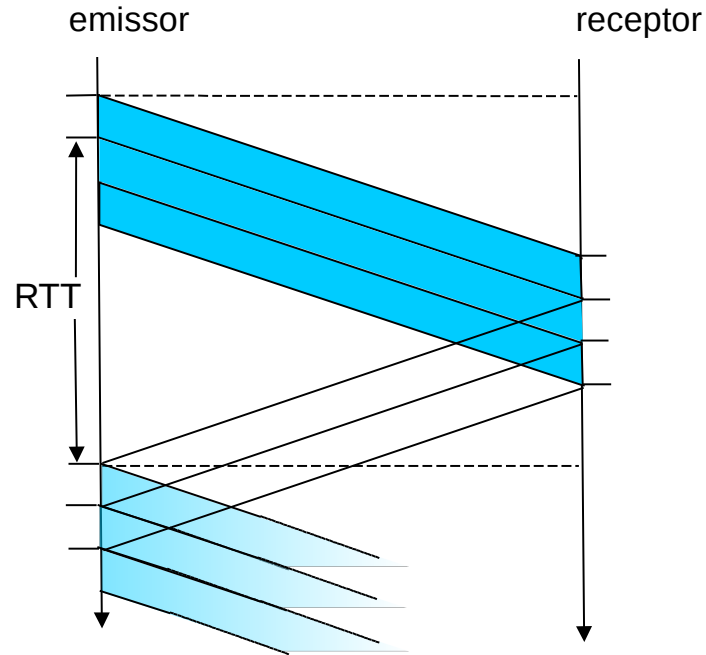
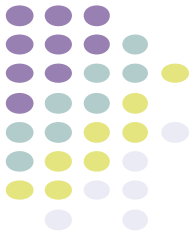
- 1KB pkt a cada 30 ms -> 33KB/seg vazão em um link de 1 Gbps

Transferência Confiável de Dados



$$U_{\text{emissor}} = \frac{L / R}{RTT + L / R} = \frac{.008}{30.008} = 0.00027$$

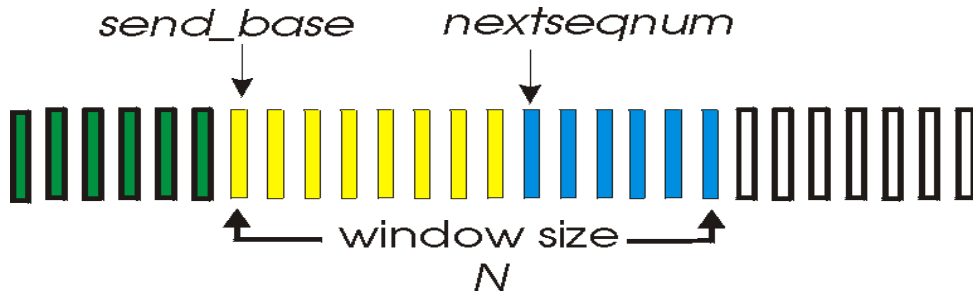
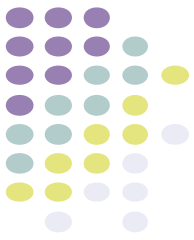
Transferência Confiável de Dados - Paralelismo



$$U_{\text{sender}} = \frac{3 * L / R}{RTT + L / R} = \frac{.024}{30.008} = 0.0008$$

Aumento da utilização
em fator de 3!

Go-Back-N



already
ack'ed

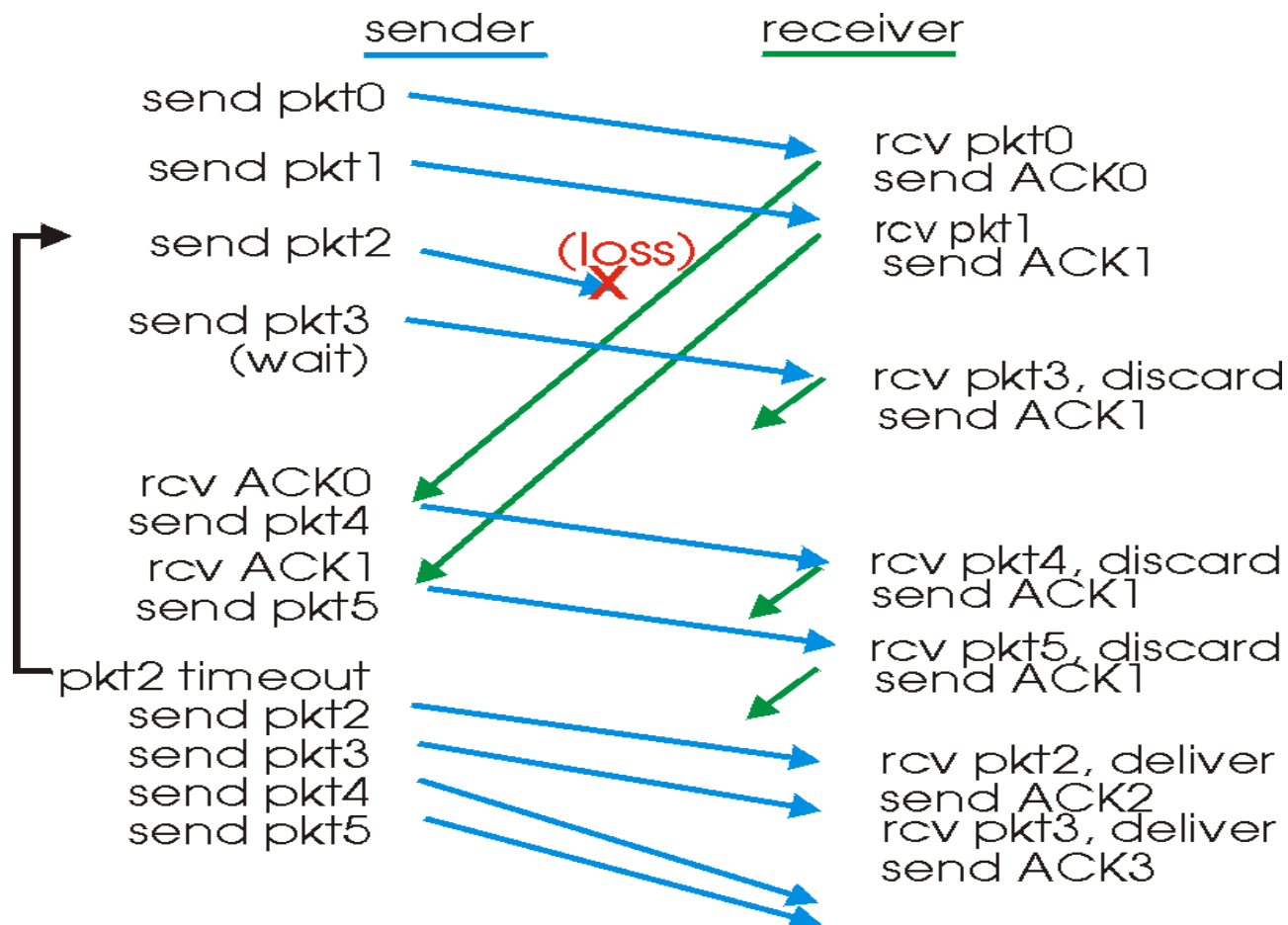
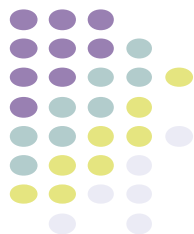
sent, not
yet ack'ed

usable, not
yet sent

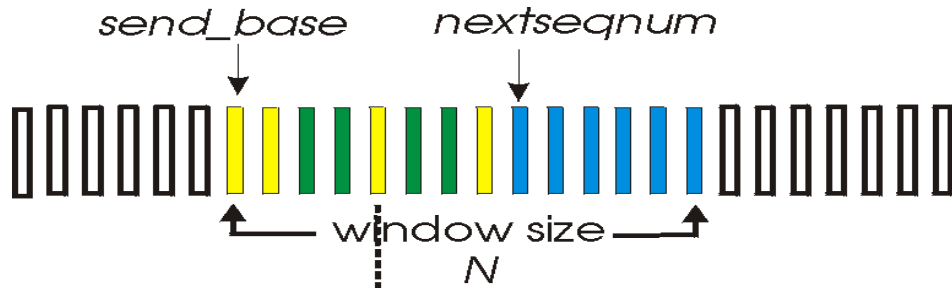
not usable



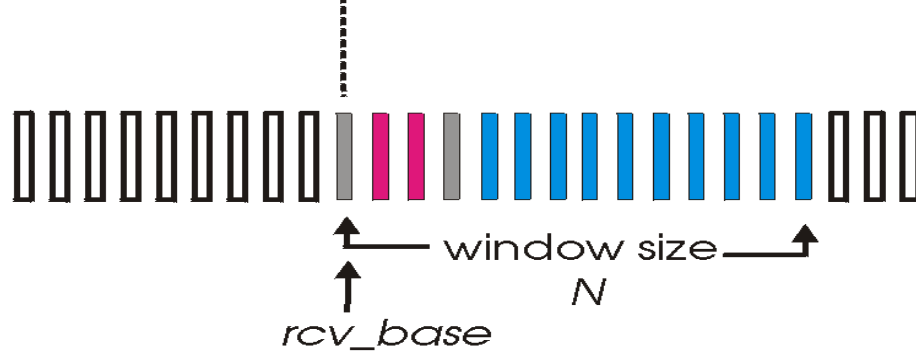
Go-Back-N



Selective Repeat



(a) sender view of sequence numbers



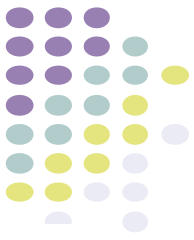
(b) receiver view of sequence numbers

 already
ack'ed
 sent, not
yet ack'ed

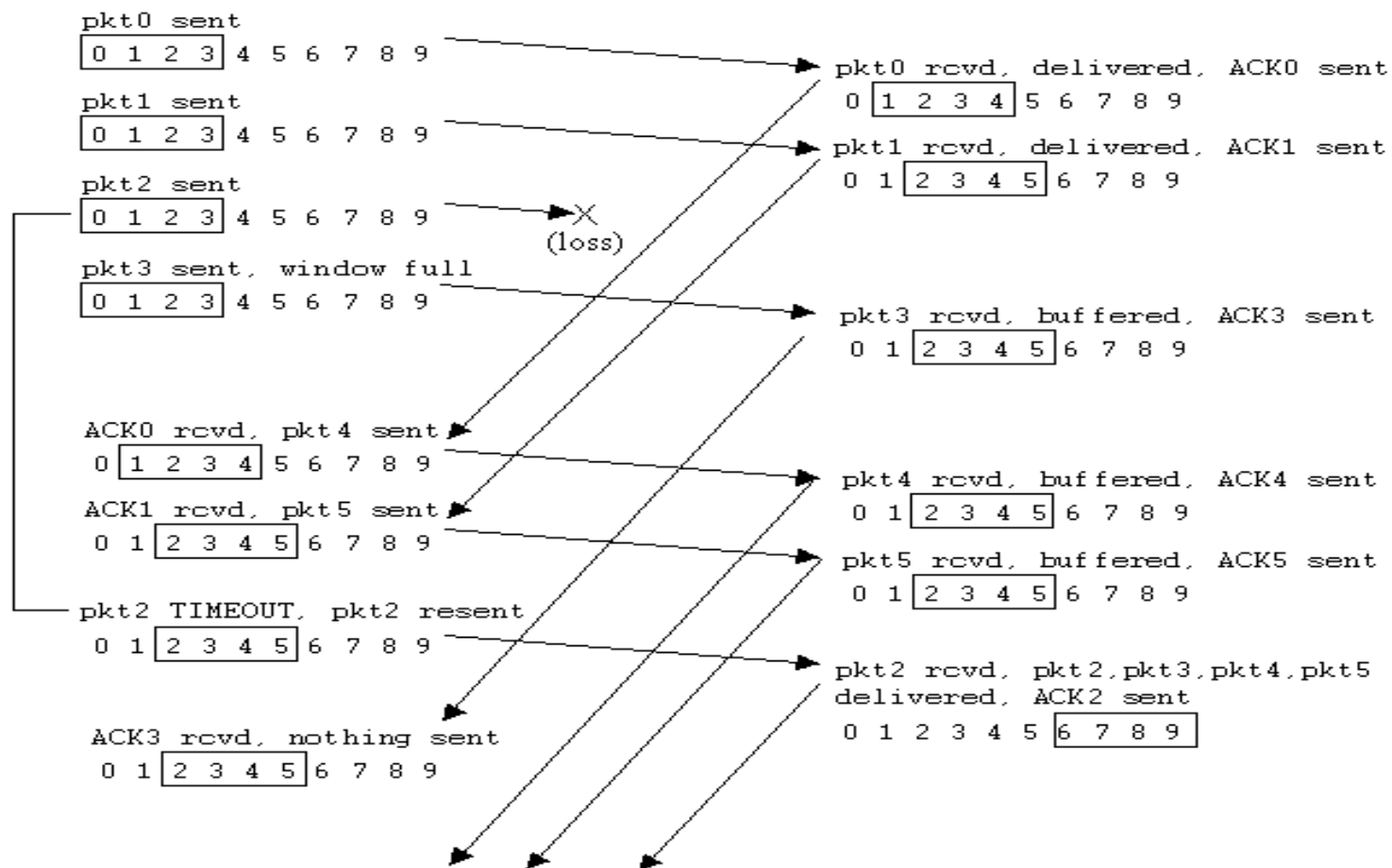
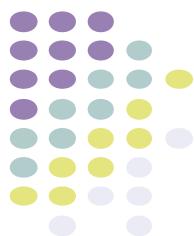
 usable, not
yet sent
 not usable

 out of order
(buffered) but
already ack'ed
 Expected, not
yet received

 acceptable
(within window)
 not usable



Selective Repeat

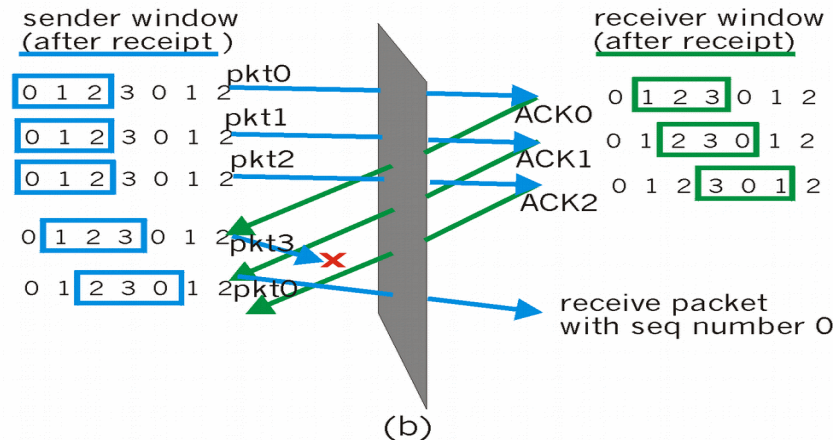
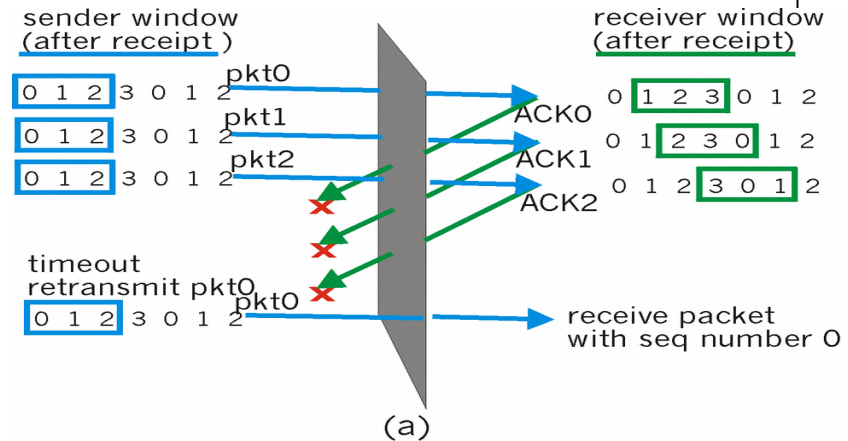


Selective Repeat

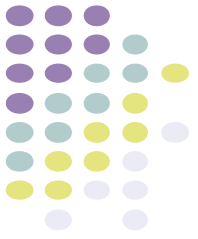
Exemplo:

- # s seq : 0, 1, 2, 3
- tamanho da janela=3
- receptor não vê diferença em ambos os casos
- dado duplicado passado em (a) e recebido como novo dado no receptor

Q: Qual a relação entre o número de sequência e o tamanho da janela?



Referências Bibliográficas



- KUROSE, J. F.; ROSS, K. W. **Redes de Computadores e a Internet: Uma abordagem Top-Down**. 6a ed., Pearson, 2013, São Paulo, SP.